

Embedded Vision Intelligent Laboratory

AUTUMN 2026 | MICROPROCESSOR SYSTEMS LAB

Lecture 5: 微算機系統實驗平台 通訊協定、感測器原理與應用

Instructor: Yen-Lin Chen (陳彥霖), Ph.D.

Dept. Computer Science and Information Engineering

National Taipei University of Technology

課程大綱 (Outline)

Part 1: 通訊協定

微處理器周邊訊號傳輸基礎

通訊基礎：Serial vs. Parallel 傳輸演進

I²C Protocol：雙線同步與多主從架構

SPI Protocol：4-Wire 高速全雙工介面

UART Protocol：非同步傳輸

Part 2: 感測器與 ADC

類比訊號量測與電路實作

類比數位轉換：ADC 取樣與量化原理

MCP3008 實作：10-bit SPI 晶片解析

感測器特性：線性度、靈敏度與遲滯

光敏電阻應用：LDR 分壓計算

Part 3: 視訊串流處理

高頻寬視覺感測與 GStreamer 框架

相機介面：MIPI CSI-2 高速差分與 I²C 控制

硬體 ISP 機制：RAW Bayer 解碼與色彩校正

GStreamer 框架：NVMM 零拷貝管線與硬體加速

OpenCV 整合：Python 串接即時相機視訊串流

為什麼要學習本章？

學習原因：把程式連上真實世界

- 讓程式讀取光線等環境變化
- 依接腳與傳輸需求選擇通訊介面
- 理解 ADC 讀值與電壓的對應關係
- 建立物聯網專題的硬體整合基礎

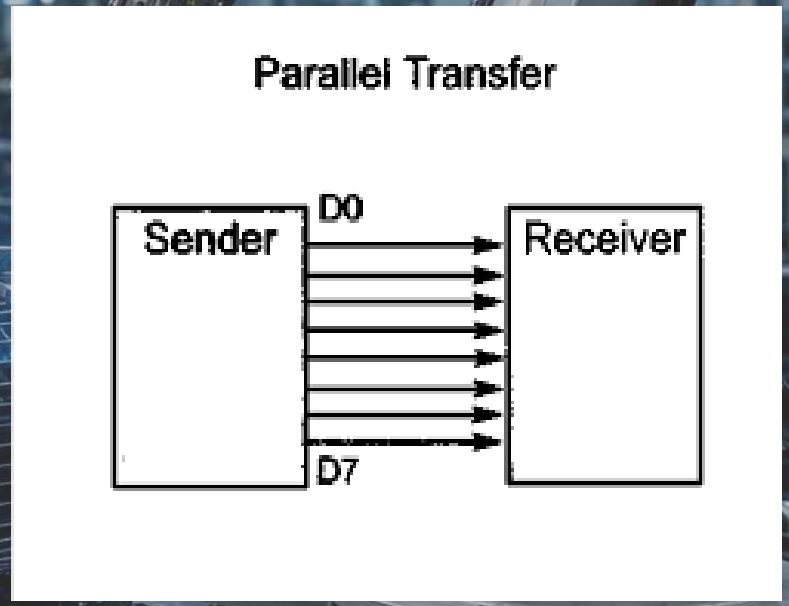
學完之後：能完成哪些任務

- 比較 I²C、SPI 與 UART 的適用情境
- 透過 SPI 讀取 MCP3008 的感測資料
- 計算 LDR 分壓與預期 ADC 讀值
- 設定亮暗門檻，作為自動夜燈的判斷依據
- 熟悉 CSI 相機和 Gstreamer

通訊基礎：串列 vs 並列

基本定義

- **並列通訊 (Parallel):** 多條通道同時傳輸多個位元 (如 8-bit, 16-bit)。像多線道高速公路。
- **串列通訊 (Serial):** 單一通道依序傳輸位元。像單線道但車速極快。
- 現代趨勢：Serial 已逐漸取代 Parallel (如 SATA 取代 IDE, PCIe 取代 PCI)。



為什麼現代通訊以 Serial 為主？

成本降低

減少接腳數 (Pin Count) 與線材成本，PCB 佈線更簡單，晶片封裝更小。

速度突破

Parallel 容易產生干擾 (Crosstalk) 與時鐘偏移 (Skew)。

Serial 透過差分訊號 (Differential Signaling) 可達更高頻率。

長距離傳輸

Serial 通訊較不受線路雜訊影響，適合長距離與複雜環境 (如 Ethernet, USB)。



I²C 通訊協定

Inter-Integrated

Circuit

I²C (Inter-Integrated Circuit) 簡介

歷史與發展

- 由 Philips (現 NXP) 於 1980 年代開發。主要用於連接低速週邊裝置（如感測器、EEPROM）至主機板或嵌入式系統。
- 自 2006 年起免專利費，成為業界標準。

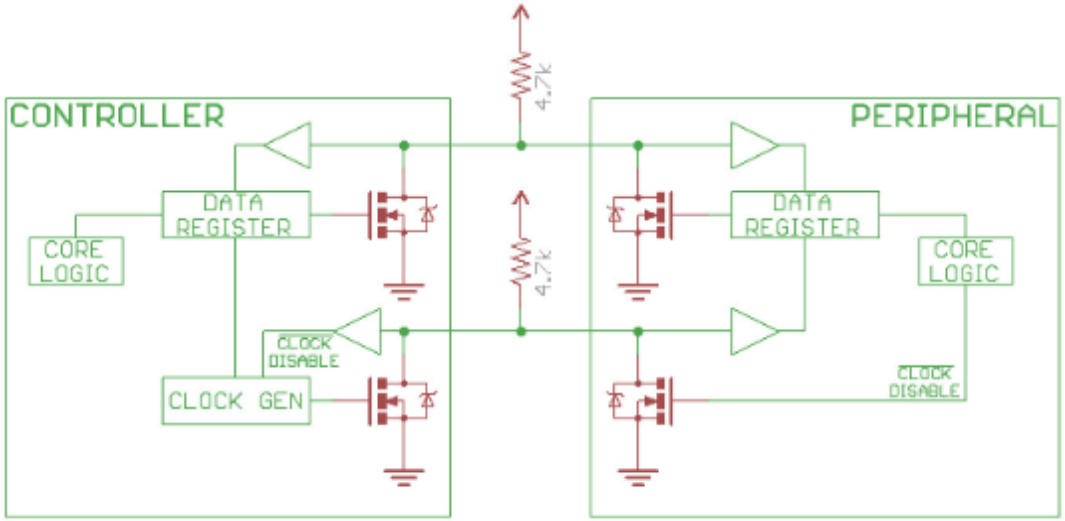
核心特性

- **雙線式 (2-Wire):** 僅需兩條訊號線。
- **多主從架構 (Multi-Master):** 匯流排上可有多個 Master 與多個 Slave。
- **同步傳輸:** 依賴時脈訊號 (Clock) 進行同步。

I²C 硬體架構

訊號線定義

- **SDA (Serial Data Line):** 雙向資料傳輸線。
- **SCL (Serial Clock Line):** 時脈線，由 Master 產生。
- **Open-Drain (開汲極):** 線路預設為 High (透過上拉電阻)，裝置只能將線路拉低 (Low)。這避免了短路風險。



I²C 通訊流程與 ACK / NACK

1 Start	SCL 為 High 時，SDA 由 High 轉為 Low。
2 位址與讀寫方向	常見為 7-bit 位址加 R/W。10-bit 定址使用多個位元組。
3 ACK / NACK	每 8 個位元後安排第 9 個時脈。接收方拉低表示 ACK。
4 資料傳輸	資料通常以 8 bits 為一組。主機讀取最後一組後回覆 NACK。
5 Stop	SCL 為 High 時，SDA 由 Low 轉為 High。

Address (7 bits) + R/W (1 bit) + ACK (1 bit) + Data (8 bits) + ACK (1 bit)...

I²C 傳輸速率 (Speed Modes)

模式 (Mode)	速率 (Bitrate)	備註
Standard Mode (Sm)	100 kbit/s	最常見，早期標準
Fast Mode (Fm)	400 kbit/s	廣泛支援
Fast Mode Plus (Fm+)	1 Mbit/s	驅動能力更強
High Speed Mode (Hs)	3.4 Mbit/s	較少見
Ultra Fast Mode (UFm)	5 Mbit/s	單向傳輸

最新趨勢：I3C (Improved I²C)

MIPI Alliance 於近幾年推出的 I²C 繼任者，向下相容 I²C 但解決了其痛點。

12.5 Mbps (SDR)

IBI In-Band Interrupts

Dyn Dynamic Addressing

I3C 支援動態位址分配 (解決位址衝突) 與帶內中斷 (不需額外 GPIO 線)，是現代 IoT 感測器的新標準。



SPI 通訊協定

Serial Peripheral Interface

SPI (Serial Peripheral Interface) 簡介

基本概述

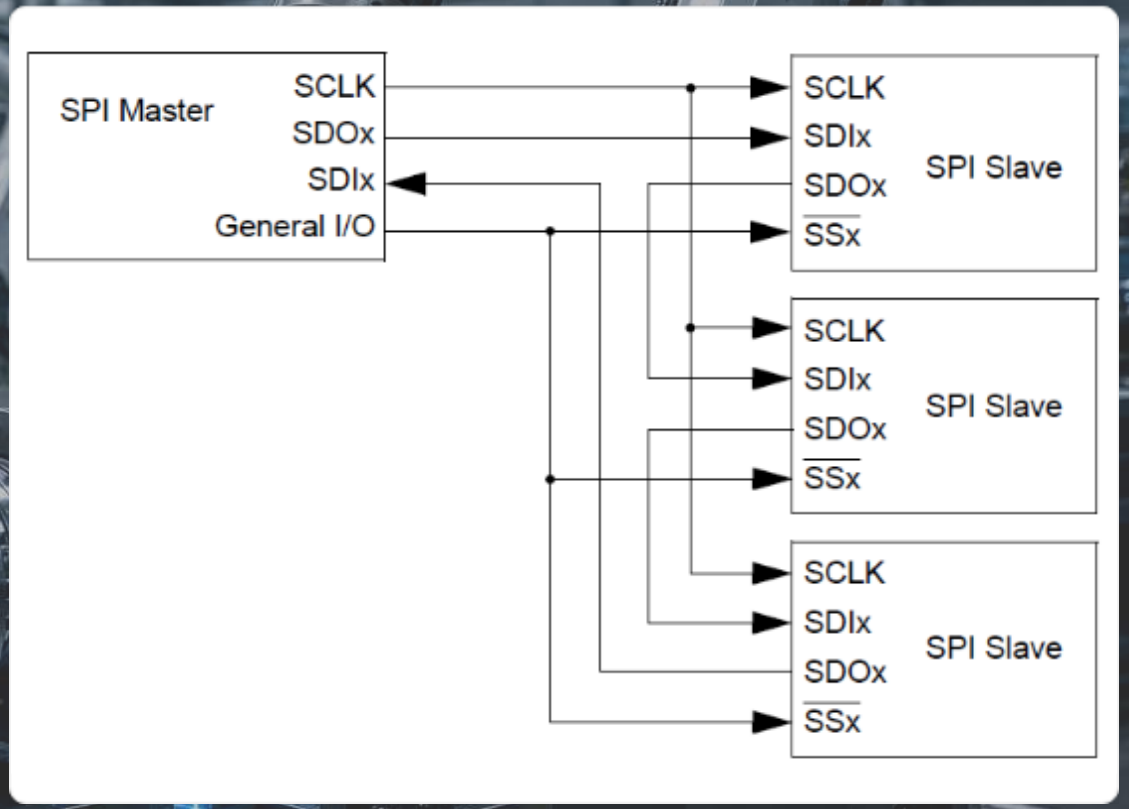
- 由 Motorola 於 1980 年代開發。是一種高速、全雙工、同步的通訊協定。
- 常見於 SD 卡、LCD 顯示器、Flash Memory 等需要較高傳輸量的裝置。

架構特點

- **4-Wire:** 需四條訊號線。
- **Master-Slave:** 單一主機，多個從機。
- **Full Duplex:** 傳送與接收可同時進行。
- **Chip Select:** 透過獨立的 CS 線選擇裝置。

SPI 硬體接腳定義

- **SCLK (Serial Clock):** 時脈，由 Master 輸出。
- **MOSI (Master Out Slave In):** Master 資料輸出。
- **MISO (Master In Slave Out):** Slave 資料輸出。
- **CS/SS (Chip Select / Slave Select):** 片選訊號，Active Low (低電位有效)。每個 Slave 需要一條獨立的 CS 線。



SPI 傳輸模式 (Modes)

SPI 依據時鐘極性 (CPOL) 與相位 (CPHA) 共有四種模式。Master 與 Slave 必須設定一致才能通訊。

Mode	CPOL (Clock Polarity)	CPHA (Clock Phase)	描述
0	0 (Idle Low)	0 (First Edge)	最常用。Rising edge 取樣。
1	0 (Idle Low)	1 (Second Edge)	Falling edge 取樣。
2	1 (Idle High)	0 (First Edge)	Falling edge 取樣。
3	1 (Idle High)	1 (Second Edge)	Rising edge 取樣。

現代 SPI 變形：速度的追求

Dual SPI

MOSI 與 MISO 變成兩條雙向資料線 (IO0, IO1)，頻寬加倍。

Quad SPI (QSPI)

使用 4 條資料線 (IO0-IO3)。常見於嵌入式系統連接 Flash Memory，執行 XIP (Execute in Place)。

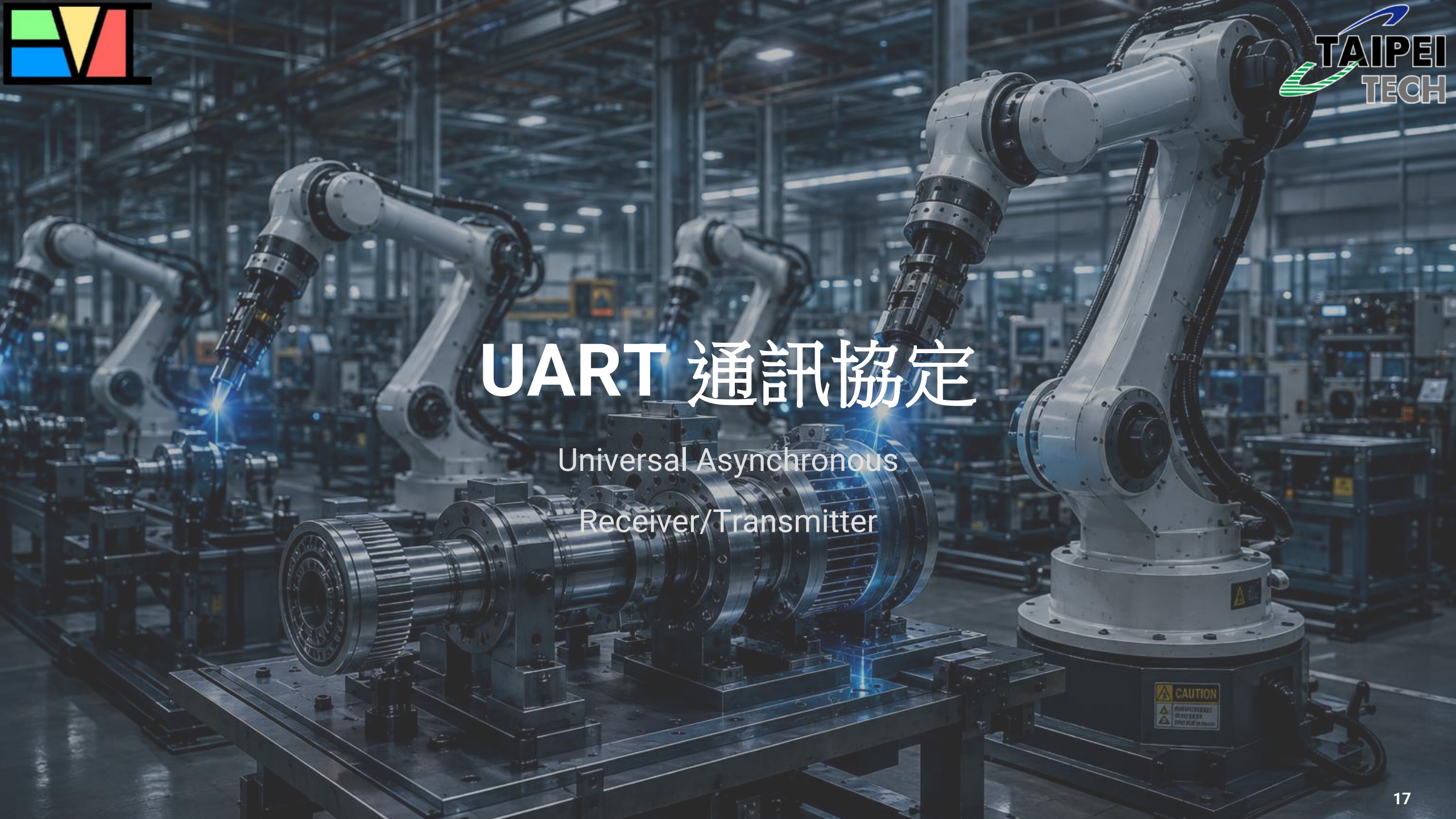
Octal SPI

8 條資料線，接近並列傳輸的頻寬，但保持較低的接腳數。

I²C 與 SPI：接線與選擇

比較項目	I²C	SPI
訊號線	SDA、SCL，共 2 條	SCLK、MOSI、MISO，加各裝置 CS
選擇裝置	使用匯流排位址	以 CS 選擇裝置
資料方向	共用 SDA，分時收發	典型四線式可同時收發
速率條件	依模式與匯流排負載設定	依晶片時序與佈線條件設定
適合情境	多個低速感測器、EEPROM	ADC、顯示器、Flash

接線數不含電源與 GND。選擇介面時，也要確認裝置支援與驅動程式。



UART 通訊協定

Universal Asynchronous

Receiver/Transmitter

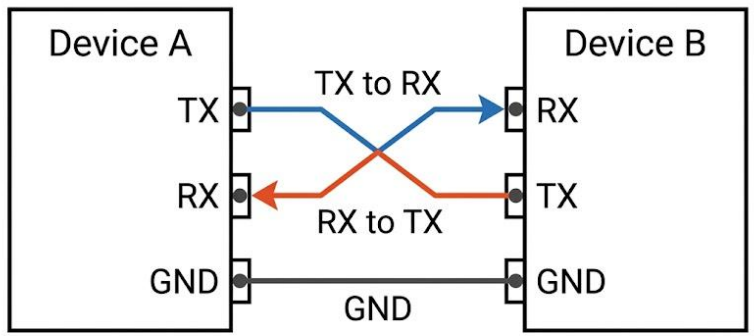
UART 簡介

非同步通訊

- **Asynchronous:** 沒有時脈線 (Clock Line)。
- 雙方必須約定好傳輸速率 (Baud Rate) 才能正確解碼資料。

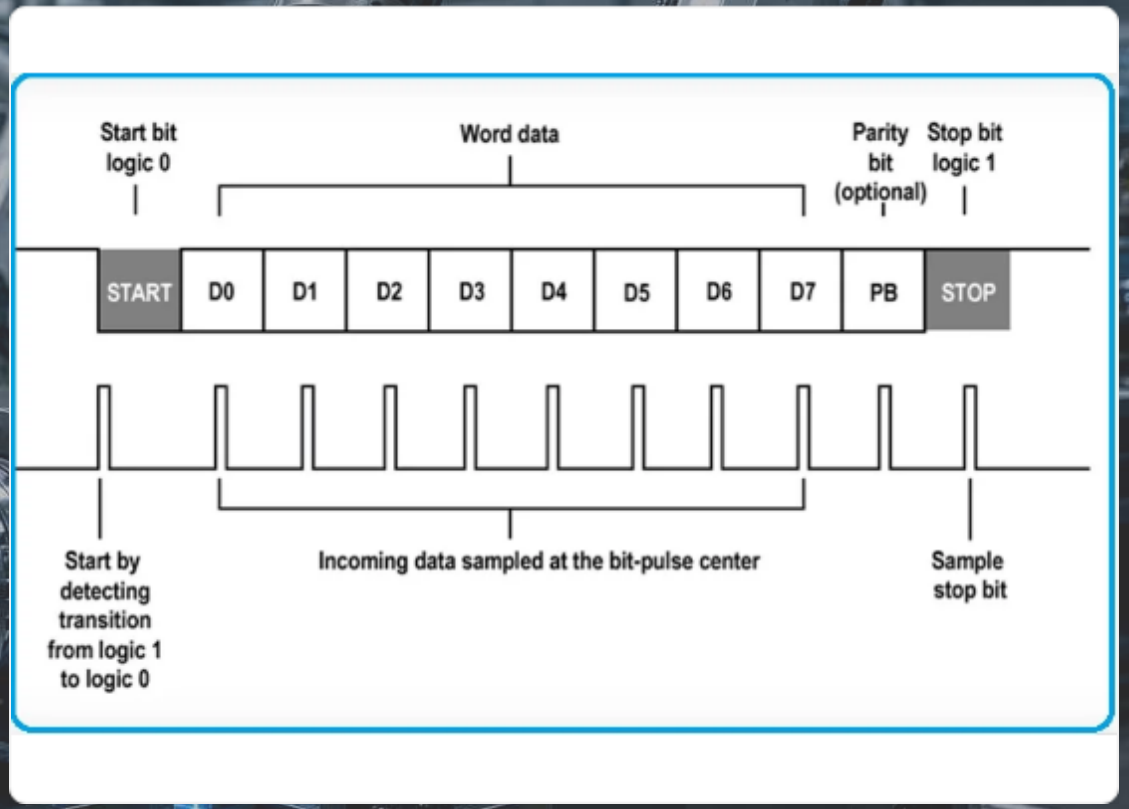
點對點傳輸

- 通常用於兩個裝置之間的一對一連接 (TX 接 RX, RX 接 TX)。
- 應用：Debug Console (除錯介面)、藍牙模組 (HC-05)、GPS 模組。



UART 資料封包 (Data Frame)

- **Idle:** 線路預設為 High。
- **Start Bit:** 拉低 (Low) 1 bit，通知開始傳輸。
- **Data Bits:** 通常 8 bits (LSB first)。
- **Parity Bit:** (選用) 奇偶校驗位，用於錯誤檢查。
- **Stop Bit:** 拉高 (High) 1 或 2 bits，表示結束。



I²C、SPI、UART：應用選擇

比較項目	I²C	SPI	UART
時脈	共用 SCL	由主機提供 SCLK	無獨立時脈線
訊號線	SDA、SCL	SCLK、MOSI、MISO、CS	TX、RX
裝置選擇	匯流排位址	片選 CS	通常點對點
設定重點	位址、上拉、時脈	模式、時脈、CS	鮑率、資料位元、校驗
本課例子	數位感測器	MCP3008 ADC	除錯主控台

先看裝置提供哪種介面，再比較接腳需求、資料量與軟體支援。



ADC 類比數位轉換

Analog-to-Digital Converter

為什麼需要 ADC？

真實世界

物理量多為**類比 (Analog)** 訊號：溫度、光線、聲音、壓力。數值是連續變化的。

轉換橋樑

ADC 負責將連續的電壓訊號取樣 (Sampling) 並量化 (Quantization) 為離散數值。

數位系統

微處理器 (CPU/MCU) 只能處理**數位 (Digital)** 訊號 (0 與 1)。

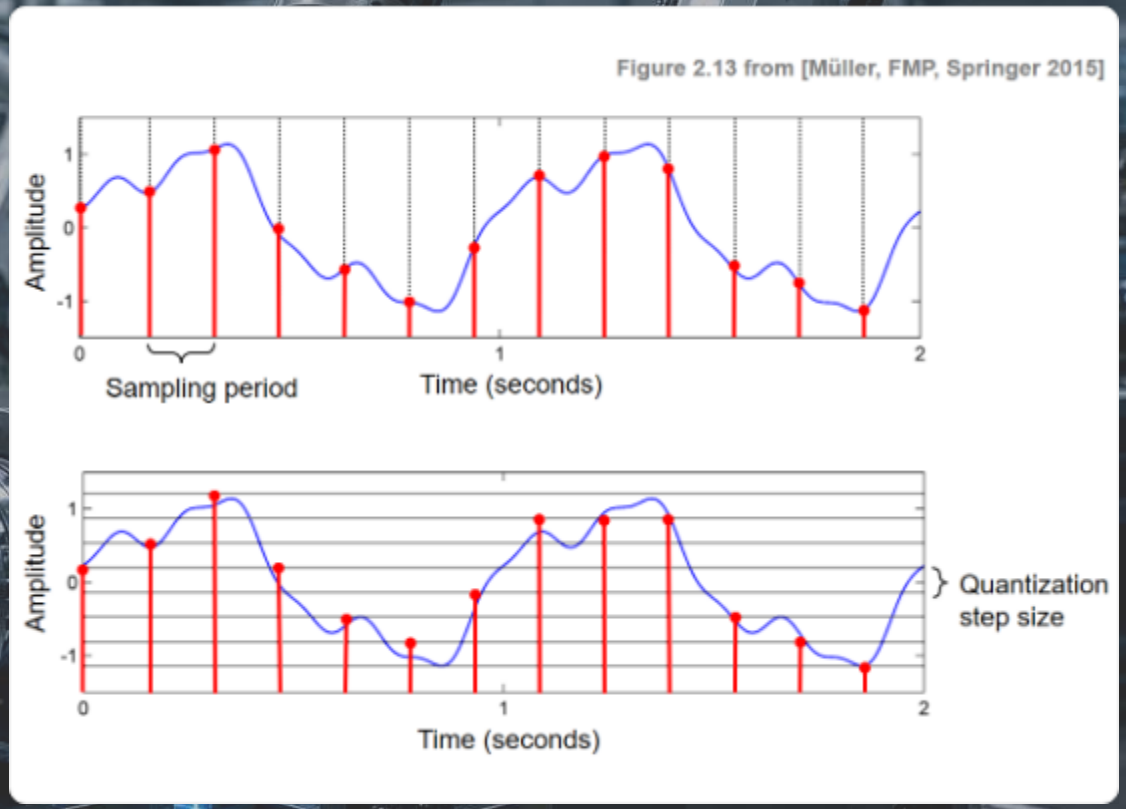
解析度 (Resolution)

- 解析度決定了 ADC 能分辨的最小電壓變化。
- 通常以 **Bits (位元數)** 表示。

$$\text{Steps} = 2^n$$

n = bit 數

- 8-bit: 256 階 (0-255)
- 10-bit: 1024 階 (0-1023) - **MCP3008**
- 12-bit: 4096 階 (0-4095)



ADC 解析度與 LSB 計算

n-bit ADC 有 2^n 個輸出碼。LSB 是理想量化步階，取決於參考電壓 VREF。

$$V_{LSB} = V_{REF} / 2^n$$

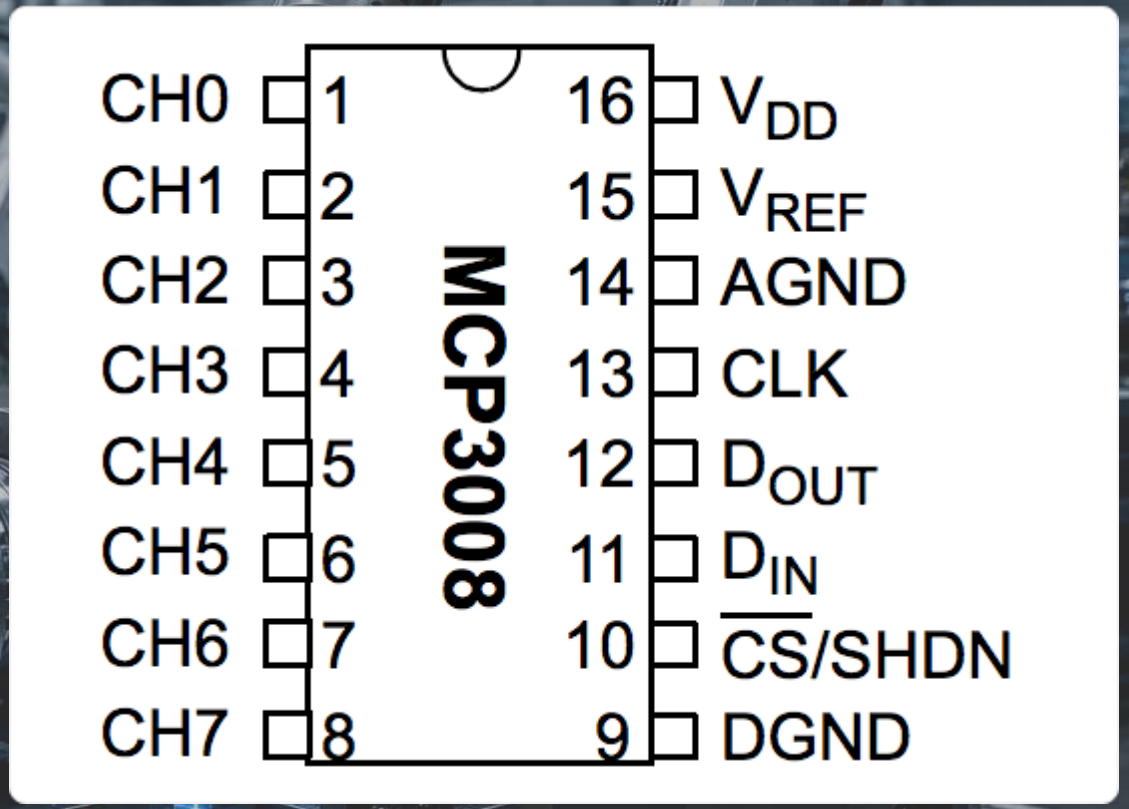
條件	計算	理想量化步階
5 V、8-bit	5 / 256	19.53 mV
3.3 V、10-bit MCP3008	3.3 / 1024	3.22 mV

較小的 LSB 表示較細的量化步階。實際準確度還受參考電壓、雜訊及 ADC 誤差影響。

MCP3008 晶片介紹

主要規格

- **解析度:** 10-bit (回傳值 0 ~ 1023)
- **通道數:** 8 個類比輸入通道 (CH0 - CH7)
- **介面:** SPI (4-wire)
- **取樣率:** 200 ksps (at 5V), 75 ksps (at 2.7V)
- **用途:** 擴充嵌入式開發版的類比訊號處理能力



MCP3008 與 TX2：接線檢查

MCP3008 接腳	用途	TX2 J21 實體腳位
VDD、VREF	供電、ADC 參考電壓	3.3 V：Pin 1
AGND、DGND	共地	GND：Pin 6
CLK	SPI 時脈	SCLK：Pin 23
DOUT、DIN	MISO、MOSI	Pin 21、Pin 19
CS/SHDN	低電位片選	CS0：Pin 24

上機前確認

本表適用 TX2 開發套件 J21，不能直接套用其他載板。
確認 SPI 腳位複用、spidev 裝置與 3.3 V 邏輯設定後，再接線。

MCP3008：SPI 封包與資料解碼

讀取 CH0，單端輸入。使用一次 3-byte 傳輸，讓 CS 在整個交易中保持有效。

傳送位元組	內容	用途
Byte 1	0x01	Start bit
Byte 2	0x80	SGL=1，通道 CH0
Byte 3	0x00	提供時脈以讀回剩餘資料

接收資料的組合方式

```
adc = ((rx[1] & 0x03) << 8) | rx[2]
```

保留第 2 個接收位元組的最低 2 bits，再接上第 3 個位元組。結果介於 0–1023。

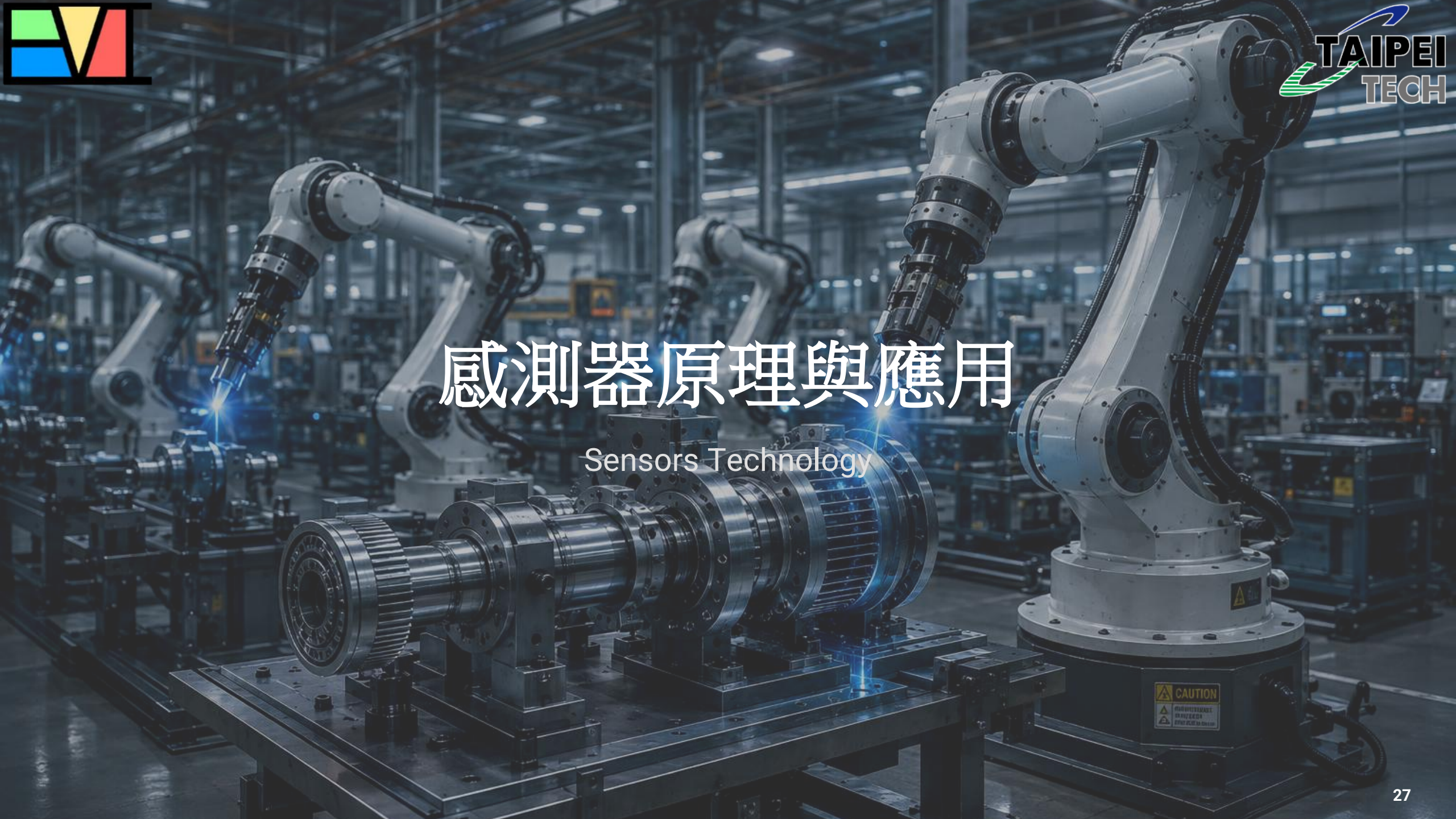
MCP3008：Python 讀取範例

先啟用 SPI 與 spidev，再依實際 /dev/spidevB.C 指定 bus、device。

```
import spidev
spi = spidev.SpiDev()
spi.open(bus, device)  # 對應 /dev/spidevB.C
spi.mode = 0
spi.max_speed_hz = 500_000

def read_adc(channel):
    if not 0 <= channel <= 7:
        raise ValueError("channel must be 0..7")
    rx = spi.xfer2([1, (8 + channel) << 4, 0])
    return ((rx[1] & 3) << 8) | rx[2]
```

使用方式：adc = read_adc(0)。結束時呼叫 spi.close()。



感測器原理與應用

Sensors Technology

什麼是感測器 (Sensor)?

"一種能偵測環境中事件或變化，並將資訊發送到其他電子裝置（如 CPU）的裝置。"

感測元件

直接感受被測量 (如溫度、光、壓力)。

轉換元件

將感測到的物理量轉換為電訊號 (如電壓、電流、電阻變化)。

訊號調節

放大、濾波、線性化 (Signal Conditioning)。

感測器分類

依輸出訊號

- **類比 (Analog):** 輸出連續電壓/電流。需 ADC (如光敏電阻, LM35)。
- **數位 (Digital):** 輸出 0/1 或通訊協定資料。內建 ADC (如 DHT11, MPU6050)。

依能量來源

- **主動式 (Active):** 需外部電源才能運作 (如霍爾感測器)。
- **被動式 (Passive):** 不需要外部電源，直接由被測物產生能量 (如熱電偶、壓電晶片)。

感測器特性：穩態與變動輸入

靜態特性

觀察輸入穩定時的量測表現。

動態特性

觀察輸入改變後，感測器如何跟隨。

線性度

輸入與輸出的關係，接近直線的程度。

響應時間

輸入改變後，輸出接近穩定值所需時間。

靈敏度

輸入改變一單位，輸出改變多少。

頻率響應

不同變化頻率下，量測的幅度與相位表現。

遲滯：同一輸入值在上升與下降過程中，可能對應不同輸出。

現代感測器趨勢：Smart Sensors

MEMS 技術

微機電系統。將機械結構微縮至晶片上 (如手機內的陀螺儀、加速度計)。

體積極小、功耗低。

IoT 融合

感測器直接整合 Wi-Fi/BLE/LoRa。支援 Matter 協定，實現萬物互聯。

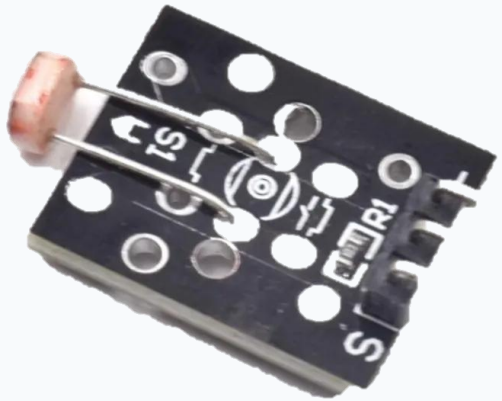
AI Edge Computing

感測器內建微處理器與 AI 模型 (TinyML)，在感測端直接判斷 (如語音關鍵字喚醒)，減少資料傳輸。

光敏電阻 LDR：光線與電阻

Light Dependent Resistor

常見 LDR 使用 CdS 等光導材料。
光線變強時，電阻通常下降。



如何連上微處理機？

LDR 改變的是電阻，ADC 量測的是電壓。
因此先用固定電阻形成分壓電路。

本課應用：量測亮暗，建立自動夜燈的判斷條件。

LDR 工作原理

物理機制

當光子撞擊半導體材料時，能量被電子吸收。

原本束縛的電子被激發成為**自由電子**，導電性增加，電阻下降。

電阻變化

- **亮電阻:** 強光下，自由電子多 -> 電阻極小 (如 1kΩ 以下)。
- **暗電阻:** 無光時，自由電子少 -> 電阻極大 (如 1MΩ 以上)。
- **光電流:** = 亮電流 - 暗電流。

LDR 分壓電路：統一接地端配置

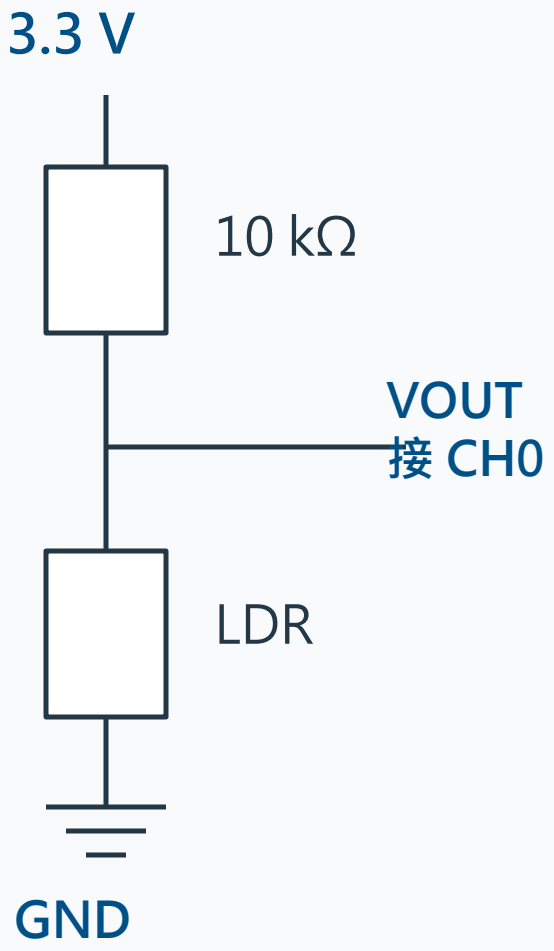
本課固定接法

VCC 與 ADC 參考電壓均為 3.3 V。
10 kΩ 固定電阻在上方，LDR 在接地端。
VOUT 接 MCP3008 CH0，所有元件共地。

分壓公式

VOUT = VCC × RLDR / (10 kΩ + RLDR)

光線變強，RLDR 下降，VOUT 也下降。
LDR 與固定電阻的位置交換後，變化方向會反轉。



LDR 計算：亮暗讀值的預期範圍

假設 VCC = VREF = 3.3 V，Rfixed = 10 kΩ，LDR 位於接地端。

計算項目	強光示例	暗處示例
假設 RLDR	1 kΩ	100 kΩ
VOUT	3.3 × 1 / 11 = 0.30 V	3.3 × 100 / 110 = 3.00 V
理想 ADC 碼估算	1024 × 0.30 / 3.3 ≈ 93	1024 × 3.00 / 3.3 ≈ 931

ADC 碼 ≈ 1024 × VOUT / VREF，輸出限制在 0–1023

表中為理想估算。實際門檻必須依手邊的 LDR 與使用環境校正。

LDR 分壓電路 — 實際計算與門檻值（Threshold）應用

【電路假設條件】 依照分壓電路設計，假設 LDR 位於接地端：

- 系統供電（V_{cc}）= 3.3V（對應 TX2 的腳位供電）
- 固定電阻（R_{fixed}）= 10kΩ
- 分壓公式：

$$V_{out} = V_{cc} \times \frac{R_{LDR}}{R_{LDR} + R_{fixed}}$$

🌞情境一：白天強光時（亮度高）強光下自由電子多，電阻極小。假設光敏電阻降至 R_{LDR} = 1kΩ：

- $V_{out} = 3.3V \times \frac{1k}{1k+10k} = 3.3V \times \frac{1}{11} \approx 0.3V$
- ADC 讀值對應（10-bit MCP3008）：約為93(數值極小)

🌙情境二：夜晚全暗時（亮度低）無光時自由電子少，電阻極大。假設光敏電阻升至R_{LDR} = 100kΩ：

- $V_{out} = 3.3V \times \frac{100k}{100k+10k} = 3.3V \times \frac{100}{110} \approx 3.0V$
- ADC 讀值對應（10-bit MCP3008）：約為930(數值極大)

亮暗門檻：校正與遲滯判斷

先量測，再設定門檻

在實際亮處與暗處各記錄多筆讀值，觀察波動範圍。
本課接法是暗處數值較高。門檻需落在兩種環境之間。

示例：使用兩個門檻，保留目前狀態

```
if adc >= 600:
    light_on = True
elif adc <= 450:
    light_on = False
# 451–599：維持 light_on 原本的狀態
```

600 與 450 是教學示例，需依量測校正。light_on 先初始化，燈具控制另接 GPIO。



CSI Camera

Camera Connector 硬體解析

- **Samtec 120-Pin 高速連接插槽**
  專門連接多通道相機卡，支援差分高速訊號與供電
- **最高 12-Lane MIPI 頻寬支援**
  彈性配置為 6 組 2-lane 或 3 組 4-lane 相機同步輸入
- **直通 Tegra SoC 晶片硬體核心**
  繞過 USB 轉接橋接晶片，達到超低延遲資料傳輸

相機模組的通訊協定



I²C 通訊協定（控制通道）

雙線式暫存器配置（SCL、SDA）

- 曝光設定：寫入快門速度與類比增益（Analog Gain）
- 模式切換：調整鏡頭解析度、幀率（FPS）與裁切
- 晶片通訊：雙向確認 ACK/NACK 確保控制指令正確

MIPI CSI-2（資料通道）

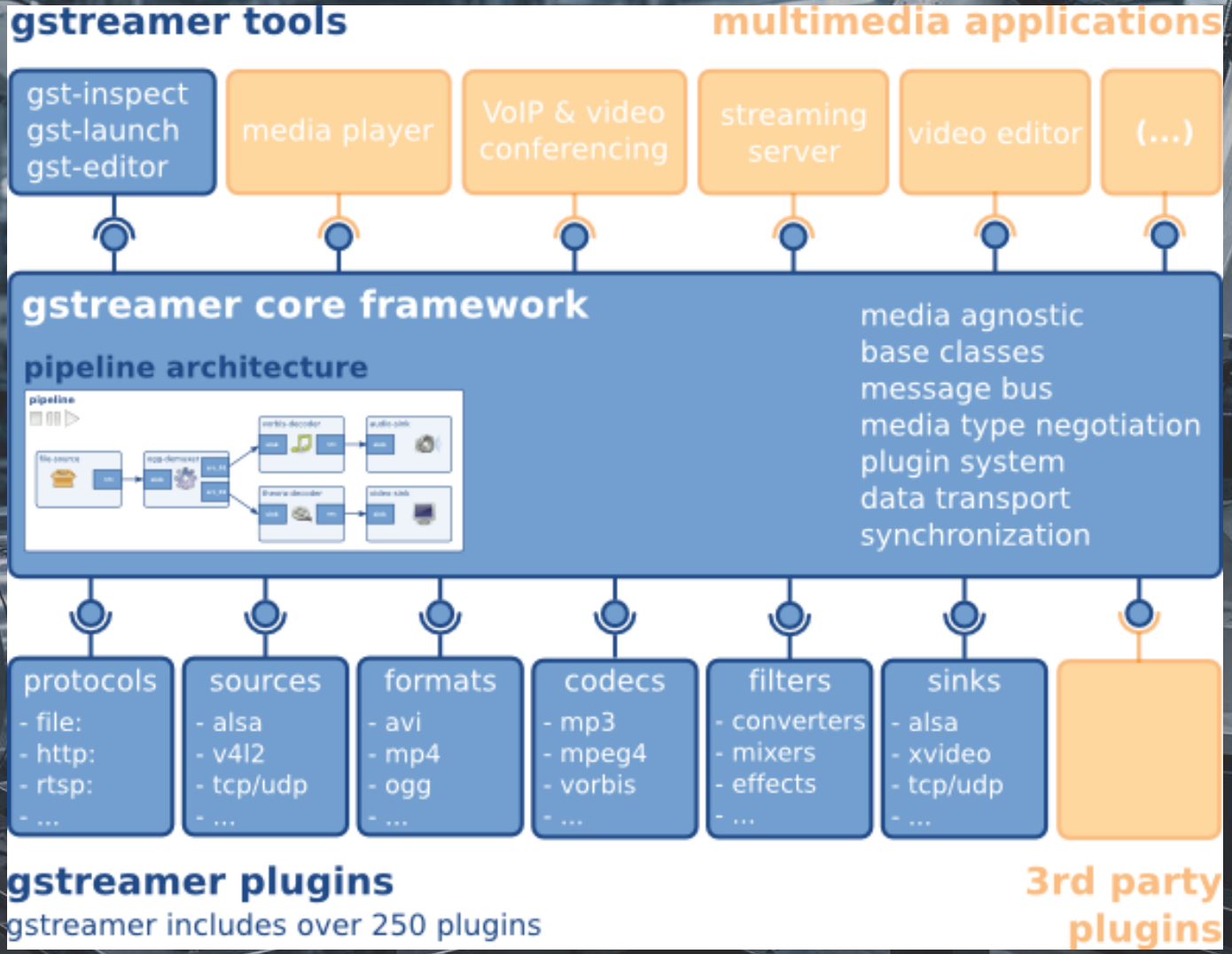
高速差分串流通道（D-PHY 訊號）

- 影像搬移：高速輸送單格感光未壓縮 RAW Bayer 點陣
- 超高頻寬：單通道達 1.5 Gbps，低電壓抗雜訊
- 硬體直達：訊號直接灌入 TX2 晶片硬體 ISP 運算

What is GStreamer?

- GStreamer 框架旨在簡化音訊、視訊或兩者兼具的應用程式的編寫。它不僅限於音訊和視頻，還可以處理任何類型的資料流。其管道設計旨在最大限度地減少應用濾波器帶來的額外開銷。這使得 GStreamer 成為設計對延遲要求極高的高階音訊應用程式的理想框架。

GStreamer


gstreamer tools
multimedia applications
gst-inspect
gst-launch
gst-editor
media player
VoIP & video conferencing
streaming server
video editor
(...)
gstreamer core framework
pipeline architecture
media agnostic
base classes
message bus
media type negotiation
plugin system
data transport
synchronization
protocols
- file:
- http:
- rtsp:
- ...
sources
- alsa
- v4l2
- tcp/udp
- ...
formats
- avi
- mp4
- ogg
- ...
codecs
- mp3
- mpeg4
- vorbis
- ...
filters
- converters
- mixers
- effects
- ...
sinks
- alsa
- xvideo
- tcp/udp
- ...
gstreamer plugins
gstreamer includes over 250 plugins
3rd party plugins

參數定義

- gst-launch-1.0 nvarguscamerasrc ! nvvidconv ! xvimagesink

  - nvarguscamerasrc：這是目前 NVIDIA 主流新版 的 CSI 相機輸入元件，基於 L4T (Linux for Tegra) 的 Camera Core 精靈進程。
  - nvvidconv：這裡沒有加參數，單純把相機輸出的 NV12/NVMM 格式轉換成普通記憶體中的標準 X11 影像格式。
  - xvimagesink：Linux 系統下一個非常傳統且基礎的 X11 視窗輸出元件。雖然效能不如上面的 nveglglessink，但非常通用，不需要額外的轉接元件（如 nvegltransform）就可以直接開出視窗。

相機的 GStreamer 完整管線架構



Source 元件

nvarguscamerasrc

呼叫 TX2 晶片 ISP 擷取影像

Caps 格式限制

memory:NVMM

鎖定 1080p、NV12 硬體記憶體

Transform 轉換

nvvidconv

硬體 VIC 負責色彩空間與畫面翻轉

Sink 輸出端

xvimagesink / appsink

螢幕零延遲渲染或送進 Python

為什麼存取 CSI 相機要使用 GStreamer ？

喚醒專屬硬體 ISP

透過 libargus 驅動 TX2 硬體 ISP，即時完成去馬賽克與色彩校正

NVMM ZERO COPY

影像停留在硬體專用記憶體，色彩轉換不經由 CPU，降低負載與發熱

非同步高效管線

自動協調訊號同步、多執行緒佇列，無縫分流至 AI 模型或網路串流